Doctrine

Un ORM taillé pour Symfony

Table des matières

I. Mise en place : 2

II. Les Entités 3

L'Entity (src/Entity/Livre.php) 4

Le Repository (src/Repository/LivreRepository.php) 4

III. La Migration 5

IV. Fixtures 5

V. Utiliser le Repository dans le Controller 7

Récapitulatif des commandes 8

Récapitulatif des notions 8

I. Mise en place :

On va utiliser ce bon vieux Docker pour se créer un environnement MariaDB 11 qu'on connaît bien. À la racine de votre projet vous allez trouver deux fichiers prévus à cet effet : un fichier **compose.yaml** et un fichier **compose.override.yaml** : ce sont deux fichiers prévus par Symfony pour y stocker une config Docker par défaut (avec PostegreSQL notamment). Nous allons donc complètement vider le fichier compose.override.yaml, et dans le fichier compose.yaml nous allons y copier ceci :

```yaml
services:
  database:
    image: mariadb:11
    environment:
      MARIADB_ROOT_PASSWORD: root
      MARIADB_DATABASE: bibliotheque
      MARIADB_USER: symfony
      MARIADB_PASSWORD: symfony
    volumes:
      - database_data:/var/lib/mysql
    ports:
      - "3306:3306"

volumes:
  database_data:
```

Puis on va le démarrer avec un **docker compose up –d**

```
nicolas@MUSS:~/wsldev/projects/premier_projet$ docker compose up -d
[+] Running 3/3
 ✓ Network premier_projet_default          Created
 ✓ Volume "premier_projet_database_data"   Created
 ✓ Container premier_projet-database-1     Started
```

Nous devons maintenant faire pointer symfony sur cette base de données fraîchement crée.

Pour cela nous allons créer un fichier **.env.local** à la racine et y coller cette chaîne :

```
# .env.local
DATABASE_URL="mysql://symfony:symfony@127.0.0.1:3306/bibliotheque?serverVersion=mariadb-11.0.0"
```

On y retrouve l'adresse de **localhost** (127.0.0.1), mais aussi les identifiants spécifiés dans le compose.yaml ainsi que le nom de la base, dans l'ordre :
SGBD://username:motDePasse@Host:Port/NomDeLaBase?parametresDivers

Un mot sur les fichiers .env

Pourquoi on en a 4 ?
Chaque fichier .env va correspondre à un environnement précis et surtout, chaque fichier .env **écrase** ou **surcharge** le prochain, dans cet ordre du plus important au moins important :

.env.local > .env.test> .env.dev> .env

L'intérêt est le suivant : Chaque fichier contient des règles générales par rapport à l'environnement, mais en étant de **plus en plus spécifique**. On a donc le fichier :

- .env qui contient des choses très généralistes comme l'URL de base du projet et sa valeur d'environnement (dev). Il est commun à tous les environnements. Il sert aussi de **documentation** pour les variables à définir dans l'appli, et est suivi sur Git.
- .env.dev n'est chargé que lorsque la valeur APP_ENV est égale à « dev ». Il remplace les valeurs qui y sont définies par les valeurs attendues en environnement de développement, et est suivi sur Git.
- .env.test n'est chargé que lorsque APP_ENV est égale à « test ». Il remplace les valeurs qui y sont définies par les valeurs attendues en environnement de test, et est suivi sur Git.
- .env.local est une surcharge **locale et personnelle**, il ne doit PAS être suivi par GIT et n'a de valeur que sur la machine où il se trouve. C'est là qu'on va mettre les **credentials**, et pas ailleurs !!! En effet on ne veut pas que des identifiants d'API etc se retrouvent visibles sur Git.

Vérifions maintenant que tout est en place : Nous allons provoquer un message d'erreur volontairement pour confirmer que la base de données est bien créé et qu'elle est accessible. Nous allons lancer la commande suivante :

```
php bin/console doctrine:database:create
```

Doctrine va alors chercher la configuration de base de données du projet courant, s'y connecter, et créer la base qu'il y a trouvé : la base **bibliotheque**. Mais vu qu'elle existe déjà, il ne peut pas la créer alors il nous renvoie ce message :

```
nicolas@MUSS:~/wsldev/projects/premier_projet$ php bin/console doctrine:database:create
Could not create database `bibliotheque` for connection named default
An exception occurred while executing a query: SQLSTATE[HY000]: General error: 1007 Can't create database 'bibliotheque'; database exists
```

II. Les Entités

Définition : Une Entité est une classe PHP qui représente la structure d'une table en base de données. Le rôle d'un ORM (Object Relational Mapping) est précisément de faire le lien entre les classes PHP et la base de données. Créons-en une pour notre projet de bibliothèque avec le **MakerBundle** :

```
php bin/console make:entity Livre
```

! Attention aux noms de classes qui doivent TOUJOURS s'écrire en PascalCase ! Une majuscule en début de chaque nouveau mot, pas de caractères spéciaux, pas de tirets ou autres !

Cette commande va vous permettre de saisir vos champs un par un dans le terminal, avec à choix fois un type string 255 non NULL par défaut.

Saisissez vos champ un par un, puis quand vous êtes arrivés à la fin, appuyez sur Entrée sans rien saisir pour valider.

- titre : string 255 NOT NULL
- auteur : string 255 NOT NULL
- annee : integer NOT NULL
- genre : string 100 NOT NULL
- resume : text NOT NULL

Une fois que c'est fait, allons voir un peu le résultat : Deux fichiers sont crées, une **Entity**, et un **Repository**.

L'Entity (src/Entity/Livre.php)

```
#[ORM\Entity(repositoryClass: LivreRepository::class)]
class Livre
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;
```

Cette classe générée automatiquement est remplie d'**annotations** qui ressemblent aux Routes que l'on a créé dans le **Controller**. Ce sont des annotations dont Doctrine va avoir besoin pour faire sa correspondance entre la base et l'objet PHP qu'il obtiendra à partir de cette classe. Pas d'inquiétude : Nous n'avons pas à les écrire à la main ! C'est **Doctrine qui s'en charge.**

Doctrine nous créé également tous les **getters** et **setters** automatiquement. Pratique.

Le Repository (src/Repository/LivreRepository.php)

Le Repository lui aussi est généré automatiquement, mais il contient beaucoup de code commenté, comme par exemple ceci :

```
//    public function findOneBySomeField($value): ?Livre
//    {
//        return $this->createQueryBuilder('l')
//            ->andWhere('l.exampleField = :val')
//            ->setParameter('val', $value)
//            ->getQuery()
//            ->getOneOrNullResult()
//        ;
//    }
```

En traduisant grossièrement ce code, on peut arriver à en comprendre la fonction : Renvoyer **UN** objet Livre ou alors NULL selon un critère de recherche determiné par « exampleField ».

En tant que tel ce code ne sert à rien, mais il nous permet de comprendre et d'affirmer la **fonction** du Repository : Il contiendra toutes les méthodes qui nous permettront de **REQUÊTER** la table Livre.

III. La Migration

À ce stade, notre structure de code est prête à fonctionner mais la table n'existe toujours pas en base de données. En effet, Doctrine ne produit les modifications en base que si on lui demande explicitement, c'est une sécurité. Un peu comme quand on fait un **Git Commit**, c'est une décision que l'on prend à un moment donnée où l'on sait que le code produit est fonctionnel.

Pour cela nous allons procéder en deux étapes :

- Premièrement, nous allons demander à Doctrine de nous générer un script de migration afin de porter les modifications en base de données avec : `php bin/console make:migration`
- Deuxièmement, nous allons dire à Doctrine d'importer ce fichier de migration en base avec la commande : `php bin/console doctrine:migrations:migrate`

Ça y est, notre table Livre est bien créée en base et possède rigoureusement la même structure que notre classe PHP ☺ Il faut maintenant la remplir

IV. Fixtures

Ce qu'on appelle une **Fixture** est un script à usage unique qui permet de **peupler** une base de données. Attention, lorsque l'on charge une fixture en base, la table est vidée : C'est comme une sorte de bouton **reset** en développement : On se prépare une base de travail propre, on l'importe, on teste / crée / modifie / supprime, et puis si on a besoin d'un départ frais on réimporte la fixture en question.

Installons le package nécessaire :

```
composer require --dev orm-fixtures
```

Puis allons modifier le fichier créé dans src/DataFixtures/AppFixtures.php

```php
<?php
// src/DataFixtures/AppFixtures.php
namespace App\DataFixtures;

// Import de la classe Livre dont nous allons avoir besoin pour construire nos objets
use App\Entity\Livre;
use Doctrine\Bundle\FixturesBundle\Fixture;
// L'ObjectManager va nous permettre de persister les données
```

```php
use Doctrine\Persistence\ObjectManager;

class AppFixtures extends Fixture
{

    public function load(ObjectManager $manager): void
    {
        $livres = [
            [
                'titre'  => 'Le Seigneur des Anneaux',
                'auteur' => 'J.R.R. Tolkien',
                'annee'  => 1954,
                'genre'  => 'Fantasy',
                'resume' => 'Un hobbit part à l\'aventure pour détruire un anneau maléfique.',
            ],
            [
                'titre'  => 'Dune',
                'auteur' => 'Frank Herbert',
                'annee'  => 1965,
                'genre'  => 'Science-Fiction',
                'resume' => 'Sur une planète désertique, un jeune noble affronte son destin.',
            ],
            [
                'titre'  => '1984',
                'auteur' => 'George Orwell',
                'annee'  => 1949,
                'genre'  => 'Dystopie',
                'resume' => 'Dans un monde totalitaire, un homme tente de résister au système.',
            ],
        ];

        foreach ($livres as $data) {
            // On crée un nouvel objet Livre à chaque tour de boucle
            $livre = new Livre();
            $livre->setTitre($data['titre']);
            $livre->setAuteur($data['auteur']);
            $livre->setAnnee($data['annee']);
            $livre->setGenre($data['genre']);
            $livre->setResume($data['resume']);

            // Et on dit à l'ObjectManager de PERSISTER le livre que l'on vient de créer
            $manager->persist($livre);
        }
        // Puis on vide l'ObjectManager
        $manager->flush();
    }
```

Voyez comme on ne précise plus les ID dans le tableau $livres ? C'est parce que Doctrine saura les gérer tout seul en base ☺ Passons la balle à Doctrine justement :

```
php bin/console doctrine:fixtures:load
```

V. Utiliser le Repository dans le Controller

Nous allons maintenant retourner du côté de notre Contrôleur LivreController.php afin de supprimer ce tableau de livres **statique** que nous avions créé dans le cours précédent. À la place, nous allons faire appel au Repository. Pour cela nous devons l'importer en haut du fichier :

```
use App\Repository\LivreRepository;
```

Ensuite, nous allons **injecter** le repository dans la fonction index(), comme ceci :

```
    public function index(LivreRepository $repository): Response
    {
        $livres = $repository->findAll();

        return $this->render('livre/index.html.twig', [
            'livres' => $livres,
        ]);
    }
```

Et dans la méthode détail :

```
    #[Route('/livres/{id}', name: 'app_livre_detail')]
    public function detail(int $id, LivreRepository $repository): Response
    {
        $livre = $repository->find($id);

        if (!$livre) {
            throw $this->createNotFoundException('Ce livre n\'existe pas.');
        }

        return $this->render('livre/detail.html.twig', [
            'livre' => $livre,
        ]);
    }
```

Les méthodes **findAll()** et **find($id)** du repository n'existent pas dans LivreRepository.php. Elles sont héritées du ServiceEntityRepository dont hérite LivreRepository, ce sont des méthodes génériques.

Récapitulatif des commandes

Commande	Rôle
docker compose up -d	Démarre la base de données
php bin/console make:entity	Crée une entité
php bin/console make:migration	Génère le fichier de migration
php bin/console doctrine:migrations:migrate	Applique les migrations
php bin/console doctrine:fixtures:load	Purge la base et charge les données de test

Récapitulatif des notions

Entité — Classe PHP qui représente une table en base de données. Chaque instance est une ligne, chaque propriété est une colonne.

Repository — Classe qui centralise toutes les requêtes à la base de données pour une entité donnée. Le contrôleur lui délègue la récupération des données.

Migration — Fichier généré par Doctrine qui décrit les modifications à appliquer à la base de données. Il permet de versionner l'évolution du schéma avec Git.

Fixtures — Classes PHP qui peuplent la base de données avec des données de test reproductibles. À n'utiliser qu'en développement.

Maker Bundle — Outil de génération de code qui crée automatiquement le **boilerplate** des entités, contrôleurs, formulaires, etc. via des commandes interactives.

BoilerPlate — On appelle le code **boilerplate** le code qui est incompressible, indispensable, mais redondant. Typiquement la structure d'une classe avec les propriétés et les getters et les setters : ça fait beaucoup de lignes à écrire, ça sera quasiment toujours les mêmes, mais on ne peut pas faire sans.

EXERCICE

1. Créez une Entity Auteur avec ces champs :

Champ	Type Doctrine	Nullable	Remarque
nom	string (100)	non	
prenom	string (100)	non	
nationalite	string (100)	non	
dateNaissance	date_immutable	non	
biographie	text	oui	Le seul nullable — un auteur peut ne pas encore avoir de bio renseignée

2. Dans notre Fixture existante, rajoutez ces données et n'oubliez pas de les **PERSISTER**

```
$auteurs = [

            [
                'nom'          => 'Tolkien',
                'prenom'       => 'J.R.R.',
                'nationalite'  => 'Britannique',
                'dateNaissance' => new \DateTimeImmutable('1892-01-03'),
                'biographie'   => 'Romancier et philologue britannique, auteur
du Seigneur des Anneaux.',
            ],
            [
                'nom'          => 'Herbert',
                'prenom'       => 'Frank',
                'nationalite'  => 'Américaine',
                'dateNaissance' => new \DateTimeImmutable('1920-10-08'),
                'biographie'   => 'Auteur de science-fiction américain, connu
pour le cycle Dune.',
            ],
            [
                'nom'          => 'Orwell',
                'prenom'       => 'George',
                'nationalite'  => 'Britannique',
                'dateNaissance' => new \DateTimeImmutable('1903-06-25'),
                'biographie'   => 'Romancier et essayiste britannique, auteur
de 1984 et La Ferme des animaux.',
            ]
        ];
```

3. Générez le script de migration avec Doctrine
4. Procédez à la migration (toujours avec Doctrine)
5. Chargez la Fixture que nous venons de faire à l'étape 2. Ça y est nos données sont en bases et prêtes à être utilisées !
6. Créez le Controller AuteurController
7. Créer la route /auteurs et lui assigner un template permettant de lister tous les auteurs. Utiliser le Repository pour trouver tous les auteurs
8. Créer la route /auteur/{id} permettant de consulter la biographie d'un auteur, et créer le template correspondant également. Utiliser le Repository pour lui passer l'auteur à afficher